



인공지능으로 문제해결하기

팀명: COdeX

팀원: 3114 최유성, 3207 오승우



목차

1. 프로젝트 개요(문제에 대한 배경과 목적)
2. 데이터 소개
3. 데이터전처리 과정
4. 평가지표 및 이론적 배경
5. 시도한 모델(1)과 결과
6. 시도한 모델(2)과 결과
7. 최종 모델
8. 프로젝트 결과 및 제언



1. 프로젝트 개요

대규모 언어 모델이 탄생하기 전부터 인공지능의 텍스트 처리 향상에 대한 요구는 항상 제기되어 왔다.

이를 위해서는 인공지능이 텍스트의 문맥을 유추할 수 있는 능력이 필요했는데, 이 문맥 유추의 기초적인 기술이 바로 감정 예측이다.

-> 모델을 구축하여 텍스트의 기초적인 감정(긍정적, 중립적, 부정적)을 예측해보자!

2. 데이터 소개

모델 학습에 사용될 데이터는 유명 SNS 중 하나인 X에서 추출한 영어 텍스트들이다.

(train: 32000개, test: 48000개)

train 데이터에선 텍스트가 내포한 감정을 중립적, 긍정적, 부정적으로 나눈 분류값이 추가적으로 기록되어있다.

X는 타 SNS와 달리 텍스트가 주된 공유 데이터이고, 텍스트의 분량도 300자 이내이기 때문에 학습에 적합하며, 분류 결과도 총 3가지이므로 평가 지표를 세우기에도 간단하다.



sentiment	info
0	Neutral
1	Positive
2	Negative

text	sentiment
@SenWarren The problem...	2
#Tweetlord is a twitter rol...	1
I just slammed my elbow i...	2
_beckett Thanks so much !	1
My gauge fell out on sup...	2
@JoannaAngel budddddy ...	2
just want to get through t...	2
Wow, look like my Top blo...	1
Disappointed	2
Ohhhhh how I miss the Br...	2



3. 데이터전처리 과정

데이터에 포함된 원문들은 다음과 비슷하게 구성되어 있다.

[@Haggis_UK](#) [@kevinhollinrake](#) *is willing to break* [#InternationalLaw](#) *like over 200* [#ToryMPs](#). *Therefore they have broken the* [#MinisterialCode](#) *and must resign!*
Simples!

[@whatwherewhenbe](#) [@hiking_hippy](#) [@zachery_nick](#) [@JoJoFromJerz](#) *This argument is so flawed here is a great explanation why, bottom line covid can be spread and the vaccine reduces your risk of getting/ spreading it to others, you can't spread pregnancy or abortions to others* <https://t.co/jBldxvfgG5> 🧐



3. 데이터전처리 과정

이 텍스트를 통해 우리 팀이 제안한 전처리 과정은 다음과 같다.

1. **멘션 제거:** @의 뒤에 이어진 텍스트는 SNS 유저를 가리키므로, 텍스트 작성자의 감정과 무관하다.
2. **URL 제거:** 인용, 동영상, 사진 등 모델 학습과 무관한 데이터를 제거한다.
3. **특수문자 유지:** 이모지(😊)와 이모티콘(XD)은 감정 예측에 핵심 역할을 하므로 유지한다.
4. **태그(해시태그) 처리:** 단순히 장소나 행사, 물체들을 나타내기도 하지만 이들이 감정을 유추할 수 있는 단서가 되어 모델의 성능에 영향을 줄 여지가 있다.
(#birthday, #gift, #accident #RIP 등...)
5. **대소문자 유지:** 일반적으로 라틴 문자로 이루어진 텍스트 데이터는 대소문자를 통일하지만, SNS에 올라오는 텍스트에선 모든 글자를 대문자로 쓰는 것도 감정을 판단하는 단서가 되므로, 대소문자를 구별할 수 있도록 유지한다.



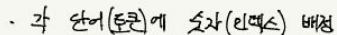
4. 평가지표 및 이론적 배경

`test` 데이터를 평가에 활용

`test`에 전처리 과정을 거쳐 모델이 예측한 값과
`test.csv`에 기록된 `label`과 비교해 정확도 측정



1) Transformer



→ "인격 인제인"

→ "위치 임베딩" : 어순

⇒ "입력+위치 벡터" : 단순 덧셈
= M_i

$$PE_z = \sin(\quad), \quad PE_{2\pi} = \cos(\quad)$$

★ **Multi-Head Attention** : 음향 자체 내 단어 관계 (vs Self-Attention) : **<가>** input-output 관계 구축
 (Transformer) Input output.

$$\begin{cases} Q = M \cdot (6 \times 6 \text{ 행렬}) \\ K = M \cdot (\quad \quad) \\ V = M \cdot (\quad \quad) \end{cases}$$
 input

변환 과정 구하기 어려움 $\Rightarrow$ 행렬 $\Rightarrow$ 스칼라 $\uparrow$
 원본에 두 개로 나누기 때문

$$\Rightarrow Q \cdot K^T \Rightarrow \frac{1}{\sqrt{d}} (Q \cdot K^T) \Rightarrow \text{softmax: 행렬 곱은 확률}$$

scaling

(Transformer) input output
 단어 간 관련성

(* head 2개 이상인 경우, 각각 계산 뒤 concatenate 후 FC layer로 최종 output)

$$\Rightarrow x \in V$$

⇒ 입력 + 위치 + 여타사항

★ Mask Attention

decoder: 아직 출력 안 된 뒤 단어는 관계없 파악 X

나 특정 단어 가운 미해 단어 처리

$\Rightarrow \text{scaling} \rightarrow \text{mask} \rightarrow \text{softmax}$



4. 평가지표 및 이론적 배경

1) Transformer

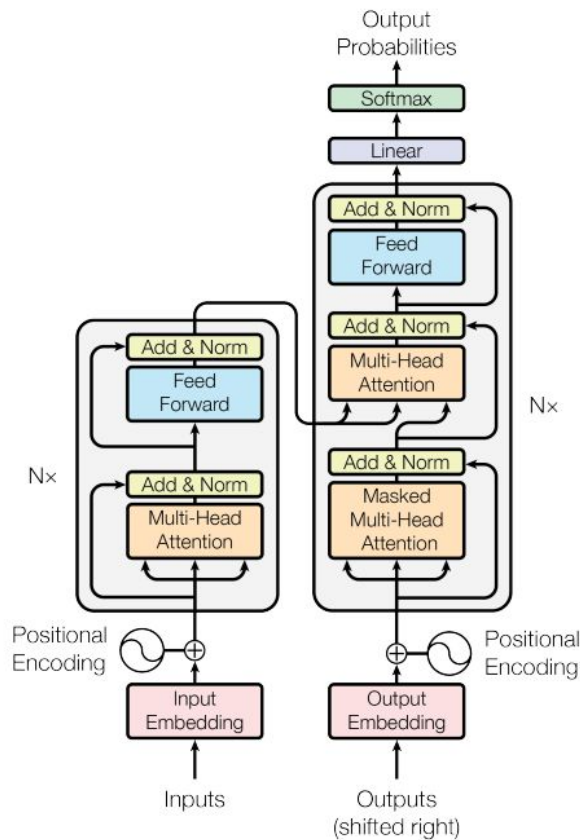
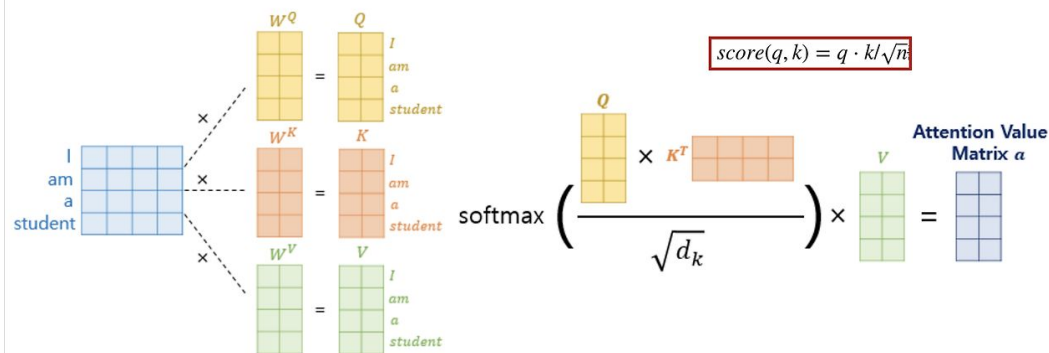


Figure 1: The Transformer - model architecture.



4. 평가지표 및 이론적 배경

2) BERT

- 기존 **NLP** 모델들은 문장을 한쪽으로만 읽는 단방향 모델이었기 때문에 문맥 정보를 충분히 반영하기가 어려웠음
- **BERT**(Bidirectional Encoder Representations from Transformers)는 Transformer 구조를 기반으로 양방향 문맥을 동시에 고려해 단어의 의미를 학습하는 **pre-trained** 모델임.
- Transformer의 Encoder 부분만을 사용함.
- 핵심 학습 방법: **MLM**(Masked Language Modeling), **NSP**(Next Sentence Prediction)



4. 평가지표 및 이론적 배경

3) RoBERTa

- BERT와 구조는 거의 동일
- BERT가 **pre-trained** 된 건 맞지만, 모든 문장들을 충분히 학습하지는 못했다는 관점에서 출발하여 더 특화시킨 버전
- 학습 데이터, 방법, 하이퍼파라미터 등을 조정해서 잠재력을 최대한 끌어냄.
- 16GB vs 160GB / **NSP** 제거 / 동적 **Masking**



5. 시도한 모델(1)

<아이디어 노트 일부 발췌>

- BERT를 사용하기로 함. (사전에 학습된 모델) -> 트위터 전용 튜닝
- 문장 내 직접 드러나 있는 **감정 관련 단어**를 포착하는 게 중요할 듯 -> **감정 단어**가 주어졌을 때 이를 **neutral / positive / negative**로 분류해서 가중치를 더 높이는 방향으로 튜닝
- ~~- BERT 파생인 RoBERTa -> RoBERTa의 트위터 특화 버전 -> 파인튜닝이 아니라, RoBERTa부터 시작해서, RoBERTa -> 트위터 특화 버전 모델부터 개발.~~
- Transformer 파생인 BERT -> RoBERTa 순서로 개발하며 자연어 처리 공부도 여유롭게 병행하자. 트위터 데이터셋을 사용하면 트위터 특화 버전이 되기 때문임.



5. 시도한 모델(1)

<기본 BERT 기반 모델>

```
import pandas as pd

import tensorflow as tf
from transformers import BertTokenizer, TFBertForSequenceClassification
```

	A	B	C
1	id	text	sentiment
2	TRAIN_00000	The problem with your massively flawed argument is one (gun	2
3	TRAIN_00001	is a twitter role playing game! Check us out at we have adora	1
4	TRAIN_00002	I just slammed my elbow into a fridge, I did not find it humer	2
5	TRAIN_00003	_beckett Thanks so much !	1
6	TRAIN_00004	My gauge fell out on superman	2
7	TRAIN_00005	buddddd that sucks about your car hope you find it soon. hi	2
8	TRAIN_00006	just want to get through the day. I have come to hate Tuesday	2
9	TRAIN_00007	Wow, look like my Top blog post is getting people new follow	1
10	TRAIN_00008	Disappointed	2
11	TRAIN_00009	Ohhhhh how I miss the Brunch	2
12	TRAIN_00010	Glad you're settling in. Will be easier too when school starts a	1
13	TRAIN_00011	Yes, Beshear is one of the best Governors that Kentucky has ev	1
14	TRAIN_00012	: What, you're not really an alcoholic? I AM V. DISAPPOINTED!	2
15	TRAIN_00013	At the Grove for 'Drag Me to Hell.' I hope that it doesn't suck.	1
16	TRAIN_00014	THE question.	2



5. 시도한 모델(1)

<기본 BERT 기반 모델>

4. 텍스트를 토큰화

```
def encode_texts(texts, tokenizer, max_len=128):  
    return tokenizer(  
        texts,  
        truncation=True,  
        padding=True,  
        max_length=max_len,  
        return_tensors="tf"
```



6. 모델 불러오기 (3-class 분류)

```
model = TFBertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=3, from_pt=True, force_download=True)
```

```
train_texts = [str(t) for t in train_texts]  
val_texts = [str(v) for v in val_texts]  
train_encodings = encode_texts(train_texts, tokenizer)  
val_encodings = encode_texts(val_texts, tokenizer)
```



5. 시도한 모델(1)

<기본 BERT 기반 모델>

```
Epoch 1/3
1800/1800 [=====] - 717s 366ms/step - loss: 0.5045 - accuracy: 0.8049 - val_loss: 0.4218 - val_accuracy: 0.8419
Epoch 2/3
1800/1800 [=====] - 649s 360ms/step - loss: 0.3389 - accuracy: 0.8751 - val_loss: 0.4302 - val_accuracy: 0.8419
Epoch 3/3
1800/1800 [=====] - 647s 360ms/step - loss: 0.2152 - accuracy: 0.9225 - val_loss: 0.4682 - val_accuracy: 0.8388
```

```
# 테스트
sample_texts = [
    "I love this game!",
    "This is terrible..",
    "It's okay, not bad."
]

print(predict(sample_texts)) # → [1, 2, 0] 예상
```

[1, 2, 2]

```
print(predict("I'm so happy"))
```

[1]



6. 시도한 모델(2)

<RoBERTa 기반 특화 버전>



```
import pandas as pd
import tensorflow as tf
from sklearn.model_selection import train_test_split
from transformers import RobertaTokenizer, TFRobertaForSequenceClassification
```

Epoch 1/3

1799/1799 [=====] - 820s 426ms/step - loss: 0.4784 - accuracy: 0.8159 - val_loss: 0.3767 - val_accuracy: 0.8530

Epoch 2/3

1799/1799 [=====] - 747s 415ms/step - loss: 0.3175 - accuracy: 0.8794 - val_loss: 0.3468 - val_accuracy: 0.8683

Epoch 3/3

1799/1799 [=====] - 746s 415ms/step - loss: 0.2442 - accuracy: 0.9103 - val_loss: 0.4004 - val_accuracy: 0.8633



6. 시도한 모델(2)

<RoBERTa 기반 특화 버전>

```
# 테스트
sample_texts = [
    "I love this game!",
    "This is terrible...",
    "It's okay, not bad."
]

print(predict(sample_texts)) # → [1, 2, 0] 예상
```

[1, 2, 2]

```
print(predict("I'm so happy"))
```

[1]



7. 최종 모델

<두 모델 간 비교>

- test.csv에 label이 없다는 사실을 깨달았을 때, 처음에는 단순히 학습 과정 시 epoch에서의 loss, accuracy 등을 통해서 비교하고자 함.

```
Epoch 1/3
1800/1800 [=====] - 717s 366ms/step - loss: 0.5045 - accuracy: 0.8049 - val_loss: 0.4218 - val_accuracy: 0.8419
Epoch 2/3
1800/1800 [=====] - 649s 360ms/step - loss: 0.3389 - accuracy: 0.8751 - val_loss: 0.4302 - val_accuracy: 0.8419
Epoch 3/3
1800/1800 [=====] - 647s 360ms/step - loss: 0.2152 - accuracy: 0.9225 - val_loss: 0.4682 - val_accuracy: 0.8388
```

```
Epoch 1/3
1799/1799 [=====] - 820s 426ms/step - loss: 0.4784 - accuracy: 0.8159 - val_loss: 0.3767 - val_accuracy: 0.8530
Epoch 2/3
1799/1799 [=====] - 747s 415ms/step - loss: 0.3175 - accuracy: 0.8794 - val_loss: 0.3468 - val_accuracy: 0.8683
Epoch 3/3
1799/1799 [=====] - 746s 415ms/step - loss: 0.2442 - accuracy: 0.9103 - val_loss: 0.4004 - val_accuracy: 0.8633
```



7. 최종 모델

<두 모델 간 비교>

- But 정량적인, 그리고 보다 정확한 **비교**를 정말 하고 싶었고, `train.csv`에서 100개만 추출하여 `test.csv`처럼 사용하고자 전략을 바꿈.

Test Results

Test Results				
Accuracy : 0.9400				
Precision: 0.9387				
Recall : 0.9400				
F1 Score : 0.9374				
	precision	recall	f1-score	support
0	0.90	0.69	0.78	13
1	0.92	0.96	0.94	23
2	0.95	0.98	0.97	64
accuracy			0.94	100
macro avg	0.92	0.88	0.90	100
weighted avg	0.94	0.94	0.94	100

Test Results				
Accuracy : 0.8500				
Precision: 0.8621				
Recall : 0.8500				
F1 Score : 0.8545				
	precision	recall	f1-score	support
0	0.56	0.69	0.62	13
1	0.83	0.87	0.85	23
2	0.93	0.88	0.90	64
accuracy			0.85	100
macro avg	0.78	0.81	0.79	100
weighted avg	0.86	0.85	0.85	100

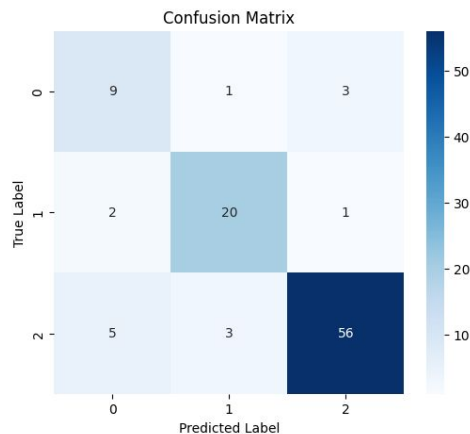
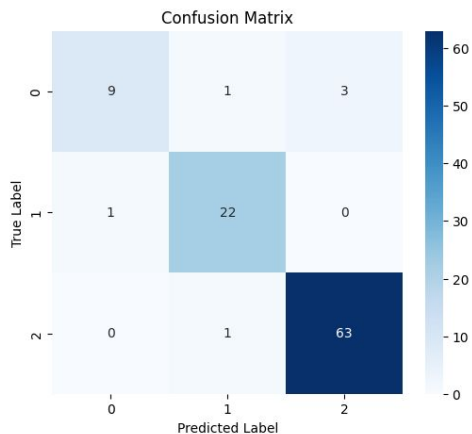


7. 최종 모델

<두 모델 간 비교>

- But 정량적인, 그리고 보다 정확한 **비교**를 정말 하고 싶었고, `train.csv`에서 100개만 추출하여 `test.csv`처럼 사용하고자 전략을 바꿈.

CM



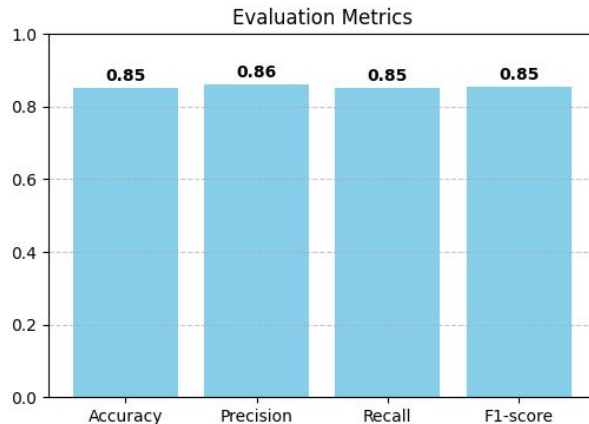
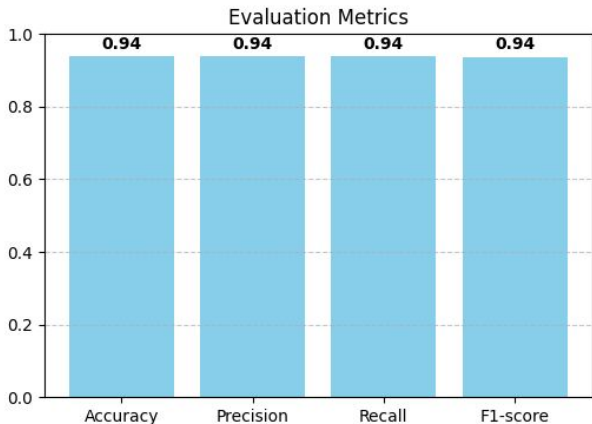


7. 최종 모델

<두 모델 간 비교>

- But 정량적인, 그리고 보다 정확한 **비교**를 정말 하고 싶었고, `train.csv`에서 100개만 추출하여 `test.csv`처럼 사용하고자 전략을 바꿈.

주요 지표 시각화





7. 최종 모델

- 비교 결과, Model 2보다 Model 1가 낫다고 판단됨.
- 즉 단순 BERT 기반 모델이 RoBERTa가 성능이 좋다는 뜻임.
- “조금” 나은 게 의미 있는 이유: 트윗 감정 분류는 문체, 풍자, 짧은 문장 등이 섞인 언어적인 **noise**가 포함된 것이므로, 성능이 조금만 올라도 실제 분류 정확도 체감은 꽤 클 것이라 판단됨.



8. 프로젝트 결과 및 제언

- 결과를 **해석**하여 **결론**을 내자. 다시 이론적 배경으로 돌아가서 성능 변화를 해석할 필요가 있다.
- RoBERTa는 원래 BERT보다 더 **큰 데이터셋 + 큰 배치에 맞게 설계**되어 있음.
따라서 본 프로젝트의 작은 트위터 감성 데이터셋 -> BERT가 더 유리
- 지금까지 공부했던 객체 인식/행동 인식을 넘어서 자연어 처리도 공부하였고, 자연어 처리 모델이 **인간의 감정을 해석**할 수 있음을 보여준 의미 있는 시도라고 생각함
- **추후 연구 방안**: 트위터 특화된 RoBERTa를 사용 & 한국어 전용 KoBERT도 사용

아이디어 노트

<https://dacon.io/competitions/official/236161/overview/description>

아이디어 노트

1. 데이터 전처리 전략

- - 멘션 제거: @username
 - URL 제거
 - 해시태그 처리: #happy → happy
 - 이모지 처리: 😊 → smiling_face
 - 소문자 변환
 - 오타/줄임말 처리: 예: u → you , gr8 → great

- 해시태그: #location이면 상관이 없지만, #emotion인 경우(= 감정이 직접적으로 드러난 경우) 해당 감정에 대한 가중치를 높이자

- URL 접속 후 맥락 보충: 나중에 시간 남으면 진행(후순위)

아이디어 노트

2. 모델 구조에 대한 논의

- API는 **HuggingFace** 사용

항목	Hugging Face + TF	Keras-NLP (TF NLP)
지원 언어	영어, 다국어, 한국어 모델 많음	영어 중심, 한국어 모델 제한적
호환성	PyTorch + TF	TF/Keras 전용
API	<code>TFBertForSequenceClassification</code> 등	<code>BertBackbone + BertClassifier</code>
pretrained 모델 수	매우 많음 (Hugging Face Hub)	제한적
장점	생태계 풍부, 다양한 토크나이저	Keras 친화적, TF Hub 연동 쉬움
학습 방식	<code>model.fit</code> 또는 Hugging Face Trainer	Keras style <code>compile + fit</code>

아이디어 노트

2. 모델 구조에 대한 논의

- 시행착오: 모델1 (HF+TF)

model compile 시 **force_download=True**와 **from_pt=True**를 하니 됨

(C) 캐시 삭제 후 재다운로드

- 캐시 경로 확인:

python

 Copy code

```
from transformers import file_utils  
print(file_utils.default_cache_path)
```

- 해당 폴더 안 `bert-base-uncased` 관련 파일 삭제 후 재시도

1 `from_pt=True` 옵션 사용

- TensorFlow용 모델을 로드할 때 PyTorch checkpoint를 TensorFlow로 변환해 불러오는 방식

python

 Copy code

```
from transformers import TFBertForSequenceClassification  
  
model = TFBertForSequenceClassification.from_pretrained  
    "bert-base-uncased",  
    num_labels=3,  
    from_pt=True # PyTorch checkpoint에서 변환  
)
```

아이디어 노트

2. 모델 구조에 대한 논의

- 시행착오: TPU했더니 안되고 GPU했더니 됨

- Transformers는 아직 Keras 3을 완전히 지원하지 않음 → `tf-keras` 필요
- 하지만 TensorFlow 2.16+(Colab 최신 버전)는 Keras 3 내장이라 `tf-keras` 설치 시 충돌

즉,

- ✓ Transformers는 tf-keras를 원하지만
- ✗ TensorFlow는 tf-keras를 싫어함

그래서 "설치하면 세션이 죽고, 안 하면 에러가 나는" 상태가 생기는 거야. ↓

아이디어 노트

2. 모델 구조에 대한 논의

단계	모델명	등장 시기	주요 학습 특징	장점	단점
1	BERT (Google)	2018	- Transformer Encoder 기반 - 양방향 문맥 학습 (Bidirectional) - MLM + NSP 학습 - 일반 텍스트(책, 위키 등) 사용	- 당시 최고 성능 - 다양한 NLP 태스크 적용 가능 - 파인튜닝으로 적은 데이터에도 강함	- 학습 데이터 규모 제한 - NSP 태스크가 꼭 필요하지 않음 - 짧은 문장/비공식 언어에 약함
2	RoBERTa (Facebook AI)	2019	- BERT 구조 유지 - NSP 제거 - 10배 이상 많은 데이터 (뉴스, 웹문서 등) - 동적 마스킹 적용 - 하이퍼파라미터 최적화	- BERT보다 전반적 성능 향상 - 다양한 도메인에 강함	- 트위터 같은 특수 도메인에는 그대로 쓰면 약간 한계
3	트위터 특화 RoBERTa (<code>cardiffnlp/twitter-roberta-base-sentiment</code>)	최근	- RoBERTa 구조 기반 - 트위터 데이터 수억 건으로 사전학습 - 해시태그, 멘션, 이모지, 줄임말 처리 강화 - 감성 분석 태스크로 파인튜닝	- 트위터 감성 분석에 최적화 - 비공식 표현, 오타, 이모지 이해 능력 우수	- 다른 도메인(뉴스, 학술문서 등)에는 성능 제한

📌 요약

- BERT: 범용 언어 이해 모델의 시작점.
- RoBERTa: BERT를 더 잘 학습시켜 성능 업그레이드.
- 트위터 특화 RoBERTa: RoBERTa를 트위터라는 특정 도메인에 맞게 최적화 → 네 프로젝트에 가장 적합.

아이디어 노트

2. 모델 구조에 대한 논의

- BERT를 사용하기로 함. (사전에 학습된 모델) -> **트위터 전용 튜닝**
- 문장 내 직접 드러나 있는 **감정 관련 단어**를 포착하는 게 중요할 듯 -> **감정 단어**가 주어졌을 때 이를 **positive / negative / neutral**로 분류해서 가중치를 더 높이는 방향으로 튜닝
- BERT 파생인 **RoBERTa** -> RoBERTa의 트위터 특화 버전 -> 파인튜닝이 아니라, RoBERTa부터 시작해서, RoBERTa -> 트위터 특화 버전 모델부터 개발.
- 따라서 **Transformer -> BERT -> RoBERTa** 순으로 “**공부**”부터 하자.

Transformer: <https://www.youtube.com/watch?v=p216tTVxues>

감사합니다

COdeX: 3114 최유성, 3207 오승우